PaN-Finder Deliverable 2.a:

Report on exploration and activities for MVP

Table of Contents

1 System Design....3
1.1 Introduction....3
1.2 Data collection....3
1.2.1 2026/01/13 Update....4
1.3 Database....4
1.4 Ingestion....6
1.5 Inference engine (Backend)....8
1.5.1 Backend API....9
1.6 User interface....10
1.7 Benchmarks....12
2 PaN-Finder System Benchmarks....12
2.1 Introduction....12
2.2 Benchmark database....13
2.3 Test Pairs....15
2.4 Metrics....15
2.4.1 General Metrics....16
2.4.1.1 Precision at K (Pc@K)....16
2.4.1.2 Context Precision at K (CP@K)....16
2.4.1.3 Reciprocal rank at K (RR@K)....17
2.4.1.4 Presence at K (Ps@K)....17
2.4.1.5 Linear Zero Position at K (lzp@K)....17
2.4.1.6 Exponential Zero Position at K (ezp@K)....18
2.4.1.7 Cumulative Gain at K (CG@K)....18
2.4.1.8 Discounted Cumulative Gain at K (DCG@K)....19
2.4.1.9 normalized Discounted Cumulative Gain at K (nDCG@K)....19
2.4.2 Metrics with 1 relevant item....19
2.4.2.1 Precision at 1K (Pc@1K)....20
2.4.2.2 Context Precision at 1K (CP@1K)....20
2.4.2.3 Presence at 1K (Ps@1K)....20
2.4.2.4 Linear Zero Position at 1K, lzp@1K....21
2.4.2.5 Exponential Zero Position at 1K (ezp@1K)....21
2.4.2.6 Discounted Cumulative Gain at 1K (DCG@1K)....21
2.4.2.7 normalized Discounted Cumulative Gain at 1K (nDCG@1K)....22

 CC BY-SA

2.4.3 Single Positive Test pair metrics....22
2.4.3.1 Context Precision at 1N (CP@1N)....23
2.4.3.2 Presence at 1N (Ps@1N)....23
2.4.3.3 Linear Zero Position at 1N (lzp@1N)....23
2.4.3.4 Exponential Zero Position at 1N (ezp@1N)....24
2.4.3.5 Linear Expected Position at 1N (lep@1N)....24
2.4.3.6 Exponential Expected Position at 1N (eep@1N)....24
2.4.3.7 zero normalized Discounted Cumulative Gain at 1N (zNDCG@1N)....25
2.4.3.8 expected normalized Discounted Comulative Gain at 1N (eNDCG@1N)....25
2.4.4 Single Negative Test Pairs Metrics....26
2.4.4.1 Absence at 1N (Ab@1N)....26
2.4.5 Extended Test Pairs Metrics....26
2.4.5.1 *Extended Presence at 1N* (EP@1N)....27
2.4.5.2 *Comparative Relative Presence @ N* (CRP@N)....28
2.4.6 Time metrics....29
2.4.7 Run metrics....29
3 Resources....32
4 Funding....33
5 Licensing....33

CC BY-SA

1 System Design

1.1 Introduction

The PaN-Finder application follows the micro-services philosophy and is composed by the following components:

1. Data collection
2. Database
3. Ingestion system
4. Inference engine (or backend)
5. User interface (or frontend)
6. Benchmarks

1.2 Data collection

Data collection is performed by a set of dedicated scripts which leverages the public APIs available at the data sources (aka participating facilities). Each script is custom tailored to the data source.

The scripts collect all the open data available at the specific data source and saved them locally in a single json file. The json file is than used by the ingestor system to populate or update the database content. The json file from each data source varies in format as the type of information that we are able to retrieve changes depending on the catalogue system used at the facility and the curation performed on the available data.

These scripts were designed and implemented to perform the first data collection while the system was been developed. They are intended to be run manually and not intended for production, although they might be used as the foundation of the data collection system that is needed for the production system.

Currently, the data collection is run as needed. We will work on defining the methodology (full or incremental) and frequency of the collection in deliverable 6.

When the project began, given the number of the possible solutions available and the speed at which the field is evolving, we decided to prioritize testing ideas and solution and focus on developing an MVP. With this decision in mind, we proceeded to collect a reasonable body of information without investing a huge amount of resources knowing that it was not complete and being aware that we had to revisit the data collection once reaching MVP and production.

The data collection follows two paths, depending on the API available at the data source and our knowledge of such system. Given our experience with SciCat, in all the data sources where SciCat is

 CC BY-SA

the data catalogue, we were able to develop scripts which leverage the full potential of the catalogue API with no further addition. If the data source uses a different catalogue system, we started by collecting all the open data entries from PaNOSC API and, then, leverage each individual data portal to further collect more information.

Examples of the data collected so far are available in the project deliverables repository which can be accessed at the following URL: https://github.com/PaN-Finder/deliverables/tree/main/task-1/data. The current version of the data collection scripts are available at https://github.com/PaN-Finder/deliverables/tree/main/task-1/scripts.

1.2.1 2026/01/13 Update

We already started the work for deliverable 6 by collecting more information on available API and extending the information collected from it. Since the first data collection, we have been collaborating with ESRF () to expand the data collected from their API. We sill start from the local PaNOSC API to retrieve the official list of open data, then we leverage their API to retrieve the complete records, together with authors, samples, datasets. We are also looking to retrieve and extract information available in the user report if present.
We are also discussing if we could include publication that are connected to the data entry, the effort required and the additional value that we will gain.

1.3 Database

The database system holds all the ingested data (body of knowledge) and the user statistics. The PaN-Finder is a RAG system, and, therefore, a vector database engine was needed. We explored multiple vector databases and decided to adopt the PostgreSql with the vector extension available in the following github repo: https://github.com/pgvector/pgvector.

We selected PostgreSql because it is a solid Sql open source database, well supported with a vast user base. It is scalable and has good performances. A docker images is also available and ready to use without any further actions required.

The PaN-Finder database has been structured in multiple tables, each with their specific functionality. Following is the list of tables and a brief explanation of their content. If more information are needed, we recommend to review the schema file present in the project repository.

- ***document***
  This table contains the document information, including the full content as it was retrieved from the data source. It also include an AI generated summary and multiple vectors created by aggregating different fields (such as title and summary) with the intent to increase the likelihood

 CC BY-SA

of a match.

- ***chunk***
  This table contains the smaller sections (chunks) of the documents properly vectorized. This table is designed to perform detailed search on the whole content of each single document which are too big to be encoded in a single vector

- ***facility***
  This is a support table containing the full name of the data sources (aka participating facilities), and their acronyms. Example: European Synchrotron Radiation Facility, ESRF.

- ***filter***
  This table contains the metadata key-value pair that have been extracted from each document. Each metadata pair can be explicit, inferred and derived. Explicit filters are key/value pairs present in explicit format directly in the document. Inferred filters are key-value pairs that have been extracted from the text of the document with the aid of an LLM and/or some level of data curation. Derived filters have been created by manipulating and further processing the other two types of filters. Derived filters are created when converting the value from string (which is the default type) to numeric, or when it is possible to extract a quantity with a unit from the simple string.
  Examples:
  - explicit filters:
    - Document: { “sample material” : “Vanadium” }
    - Filter: { “sample material” : “Vanadium” }
  - implicit filters:
    - Document: {
      “Abstract” : “… sample is human blood at body temperature. …”
      }
    - Filter: {
      “sample material” : “human blood”,
      “sample temperature” : “37 C”
      }
  - Derived filters:
    - Source filter: { “sample temperature” : “37 C” }
    - Derived filter: { “sample temperature” : { “value” : 37, “unit” : “C” }}

CC BY-SA

- ***filter_description***
  This table allows to enrich each metadata key found in the corpus of documents with a proper description and its vector representation. This functionality allows us to increase the probability to match a document with a user query through similarity search specifically on the filter keys.

- ***filter_key***
  This table contains the list of all the keys found in the corpus of documents and, for each key, its vector representation. This table is specifically designed to be able to perform similarity search on the key names directly and provide more flexibility when the user is looking for a key while a misspelling is present or different variance of the same string are present.
  Example:
  - User query: sample
  - Matching key: sample, sample temperature, sample name, saample, etc.

- ***statistic***
  This table contains all the queries submitted by the users, the components extract by the LLM, the RAG output and the execution time. This table is important to further improve the project, its benchmarks and find potential corner cases that the system is not trained or configured to handle correctly.

- ***feedback***
  This table contains the user feedback collected through the application frontend. For each result returned by the system, the frontend allows the user to provide a feedback on the actual relevancy of the result in relation to the query submitted. The feedback is binary: relevant or non relevant. If no feedback is provided, no conclusion can be drawn.

1.4 Ingestion

Once the data has been collected from the data sources and stored in files, the ingestion process can take place. The ingestion process consists of the following steps:

- ***Document insertion***
  Each document collected from data sources is inserted in the document table if it has a DOI and it is not already present. Documents without DOI are discarded.

- ***Chunk creation and insertion***
  Each document is divided in chunks of acceptable length, dictated by the LLM settings, so they can be encoded and vectorized. Each chunk with the associated encoded vector is inserted in the chunk table

 CC BY-SA

- ***Filter extraction***
  For each document, it extract all the filters key/value pairs, perform same data curation on the keys and store them as explicit keys-values in the filter table. It also insert each key and its vector in the key table.
- ***Numeric filters conversion***
  Due to data collection method combined with the use of json, all the filters key-values pairs have the value in string format. Some of the values are can interpreted as number, ranges or quantities with units. This ingestion step creates new filters entries with a single value and the associated units, if present. Such filters are stored in the same filter table as derived types.
  If the explicit entry is a range, two derived entries are created, each with the min and the max value, while the the suffix min and max is appended to the key. In case of quantities with units, the value and the unit are separated and a new derived entry is created with the numeric value and unit properly populated.
- ***Filters value conversion***
  For each filter present in the filter table, it converts the values from string to the proper type which can be boolean, numeric or timestamp. The new value is saved in the matching column of the original entry.
- ***Filter enrichment***
  This step further curate and normalize all the filter entries related data source (institute) and beamline, instrument. The new entries created are stored back in the filter table as derived types.
- ***Document summary generation***
  All the documents are submitted to the LLM to produce a 100 words summary. The summary and its vectorized version is stored along in the document record.

While developing the MVP, we have explored the use of LLM models LLAMA 3 and CHAT GPT 4.1 mini running in complete isolation. From our initial tests, they both provided acceptable results, although closed source model provided slightly better output. Performances were highly dependent on the hardware they run on, and we did not have a dedicated GPU machine ready made to be used for this project. Through our existing infrastructure at ESS, we were able to secure an instance of CHAT GPT 4.1 mini on Azure, so we decided to focus on the results and postpone to a later time, and only if needed, the investigation about running other models on different hardware, models' fine tuning and the sustainability of the project.

CC BY-SA

1.5 Inference engine (Backend)

The reference engine, or backend, is the service that provides the main search functionality once the data collected has been ingested and the database populated with body of knowledge. When the user wants to perform a search, she posts the search prompt (or user prompt) to the backend, and receives back a results set containing the documents that are relevant to the context of the search.

The backend submits the user prompt received to an LLM together with a system prompt specifically crafted to extract the three main components:

- user intention
- keywords
- filters

This components are used to build a hybrid query which is than submitted to the database.
The database produces the final result which is a set of document that the RAG system believes are relevant to the user prompt.

The following is an example of user query directly extracted from our benchmarks:

> *Look for research projects on protein structure-function relationships in human health and biotechnology that utilize X-ray crystallography and cryo-EM, with a release date set for March 29, 2027.*

From this user prompt, the following components are extracted:

- Intention: *protein structure-function relationships in human health and biotechnology*
- Keywords: *protein, structure-function, human health, biotechnology, "X-ray crystallography", cryo-EM*
- Filters:

```
{
  "logic": "AND",
  "conditions": [
  {
    "name": ["releaseDate", "dates.date", "startDate", "publicationYear", "endDate"],
    "value": "2027-03-29",
    "operator": "="
  }]
}
```

The components are than used to build an hybrid query which can be summarize as follows:

```
SELECT
doi, overall_score,
similarity_score, chunk_similarity_score, full_match_score, partial_match_score, keyword_score
FROM (
  -- Subquery 1: Document Similarity (title + summary (generated)) => similarity_score
```


CC BY-SA

```
  -- Subquery 2: Chunk Similarity (title + text chunks) => chunk_similarity_score
  -- Subquery 3: Hard Filter Matches => full_match_score, partial_match_score
  -- Subquery 4: Keywords (Full-Text Search) => keywords_score
ORDER BY
  overall_score DESC,
  full_match_score DESC,
  partial_match_score DESC,
  similarity_score DESC,
  chunk_similarity_score DESC,
  keyword_score DESC
LIMIT 10
```

When the hybrid query is submitted to the database, the results is a table with the list of relevant documents and the relative scores:

DOI	Overall Score	Keyword Score	Full Match Score	Similarity Score	Partial Match Score	Chunk Similarity Score
10.15151/ESRF-ES-1567842882	0.5148117386	0.0833333333	0.1428571429	0.0029069767	0.1428571429	0.1428571429
10.15151/ESRF-ES-1472038685	0.3412698413	0.0555555556	0.1428571429	0.0	0.1428571429	0.0
10.15151/ESRF-ES-55108289	0.3127917834	0.0588235294	0.0	0.1428571429	0.0	0.1111111111
10.15151/ESRF-ES-159994365	0.3127917834	0.0588235294	0.0	0.1428571429	0.0	0.1111111111
…	...	...	...	...	...	...

From the few results shown, we can clearly conclude that only the first couple of rows are really relevant, while the rest of the rows can be seen as “suggested” reading. This conclusion brought us to implement and apply the knee point algorithm (https://www.kaggle.com/code/kevinarvai/knee-elbow-point-detection, https://github.com/arvkevi/kneed) to our results set. With the help of this functionality, we are able to provide the user with two set of results: a relevant one which should be close to what they are looking for, and a suggested one which contains suggested reading that is less relevant but somehow still connected to their query.
Also in this subsystem, we were able to secure access to a CHAT GPT 4.1 mini on Azure, so we followed suit as with the ingestion to postpone at a later time further investigation on different models and hardware to provide a better overview on performances, costs (monetary and environmental) and project sustainability.

1.5.1 Backend API

The backend is a headless service implementing a REST API which provides the following endpoints:

- GET **/health**
  This endpoint return the over all health of the application. The information returned is:
  - status: healthy or degraded
  - message: descriptive message of the status

CC BY-SA

  - database: healthy, degraded or error
  - version: the application version
- POST **/search**
  process the user prompt and provide the set of relevant results
- POST **/search/explain**
  it provide an explanation why a specific result is relevant in the context of the user query.
- POST **/search/structured**
  It allows to specified the query components directly and bypass completely their extraction
- GET **/document/raw/{doi}**
  It returns the raw json of the document as it has been provided in the data collected from the data sources.
- GET **/document/{doi}**
  It returns title, abstract and facility of the relevant documenting
- POST **/feedback/submit**
  It collects the user feedback if a specific document is relevant or not in the context of the user search
- POST **/session/create**
  Create a user session

1.6 User interface

The user interface (or frontend) is a single page react app that runs entirely in the user browser. It provides a simple interface for the user to submit their query and review the results and read the relevancy explanation.
It also shows the user prompt components extracted by the system and allows users to provide feedback if a returned document is relevant or not for their query.

Figure 1 shows the frontend with a query submitted and the list of results already retrieved.
The frontend always show the details of the most relevant entry, together with relevancy explanation, which is LLM generated.

We are actively working improving the frontend usability by interviewing willing users and collecting their feedback.

CC BY-SA

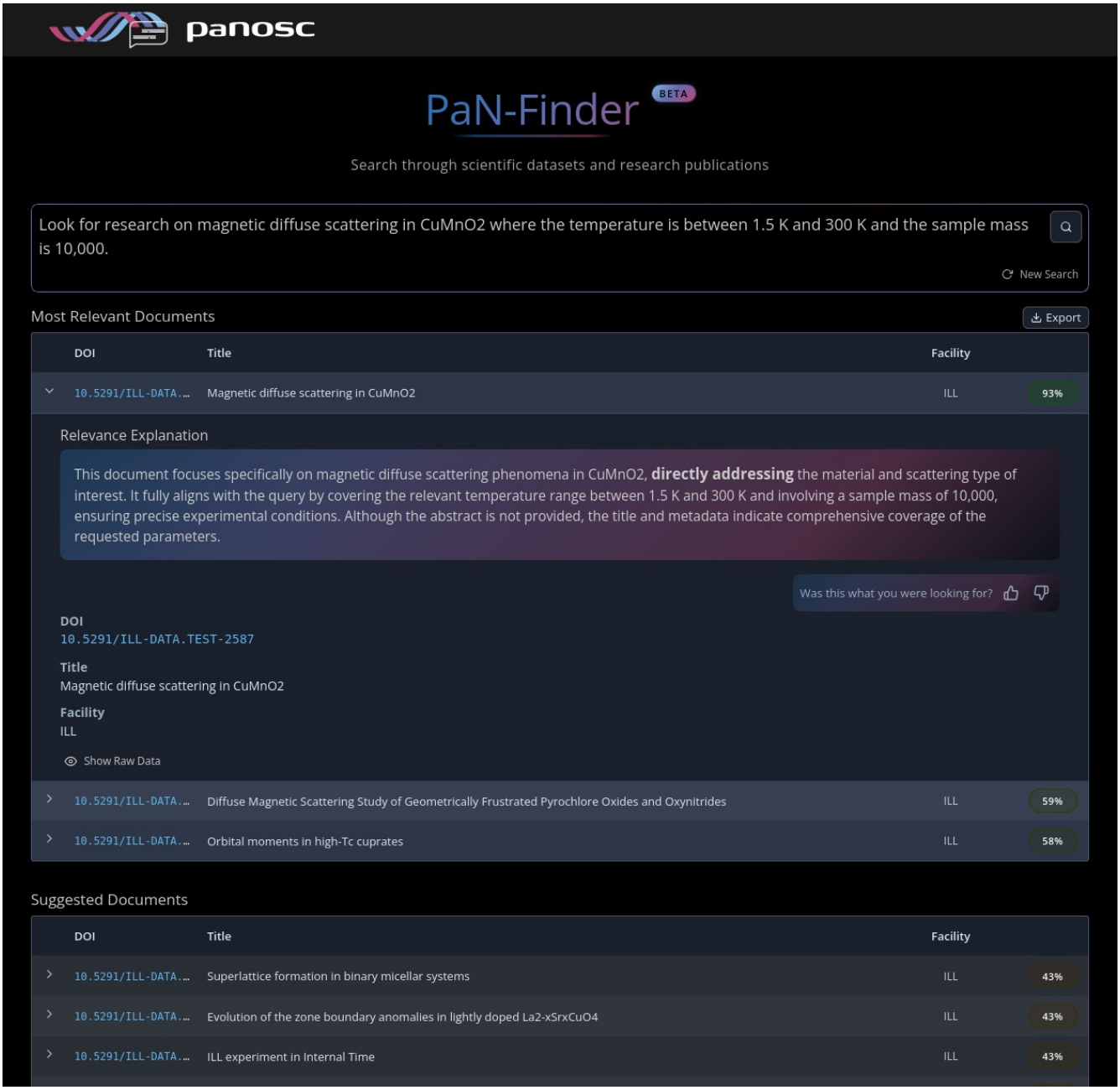

Figure 1: PaN-Finder Frontend with a query submitted and results shown.

CC BY-SA

1.7 Benchmarks

Benchmarks are used to evaluate the performance of the application. By performance, we actually mean accuracy which is the measure of how relevant the results provided are according to a set of metrics and a predefined set of query and expected answers. In order to measure accuracy, the benchmarks run with a set of predefined test pairs which contain a user prompt associated with an expected outcome. Accuracy is computed on how many of such expected outcomes are met by the system with the specified configuration.
Benchmarks are explained in great details in the section 2.

2 PaN-Finder System Benchmarks

2.1 Introduction

The PaN-Finder project has developed a benchmark system tailored to test and measure the performance and accuracy of the system, its design and components.
Every time a change is made to the system, a new run of the benchmark system is performed. A benchmark run consists of a number of settings (called hyper parameters) applied to the combined system that comprises the application and the benchmark itself. The hyper parameters includes all the tests performed, the responses of the application, the set of metrics that are computed and additional statistics collected.
In short, a benchmark run can be indicated as the following function:

$$Rn_z = R(B_i, A_j, H_v) = (Rs_z, M_z)$$

where

- Rn_z is benchmark run z
- B_i is benchmark version i
- A_j is application version j
- H_v is hyper parameters version v
- Rs_z is all the results set returned by the application
- M_z is all the metrics values collected and computed from the results sets R_v

The hyper parameters is a tuple of sets defined as follow:

$$H = (K, N, T, M)$$

where:

- H is the hyper parameters selected
- K are the K values used to compute the overall scoring from the different scoring values

 BY-SA

- N is the number of items returned in the results set at each call.
- T is the set of test pairs selected
- M is the set of selected metrics. Some of the metrics are applied to each individual test pair, some of them are computed at the run level

Each version of the application A_j will be tested with multiple benchmark runs with a selected combination of the hyper parameters values. The hyper parameters values will be explained in the following pages.
Each test pair is a tuple of three elements as follows:

$$TP = (P_u, I_T, r_e)$$

where:

- P_U is the user prompt
- I_T is the target item or items, aka the item or items that we are testing against.
  The target items are indicated by their DOI.
- r_e is the expected ranking of the item. This is the position of the item in the list of results set returned. Initially it is always set to 1 (top position), as we do not have any prior knowledge regarding the topic. We are hopeful to develop such knowledge as we collect more benchmarks runs

2.2 Benchmark database

All the information related to benchmarks are stored in a dedicated database, named pan_finder_benchmarks. This database contains the definition of each test pairs, together with information and results of each benchmark run.
Each test pair is saved in the benchmark database under the table *test_pairs* with the following schema:

- `tpid UUID PRIMARY KEY,`
- `userprompt TEXT,`
- `targetdoi TEXT,`
- `expectedRank INTEGER,`
- `promptid UUID,`
- `expertname TEXT,`
- `groupid TEXT,`
- `type test_pair_type,`
- `created DATETIME DEFAULT CURRENT_TIMESTAMP,`
- `deprecated DATETIME,`
- `type test_pair_type`
- `targetgroup text`

CC BY-SA

When a run is started, the benchmark code will load the selected test pairs indicated in the hyper parameters, submit each test pairs to the application, collect the results sets, together with execution times, and compute the individual test pairs metrics specified again in the hyper parameters. Once all the test pairs have been tested, the run metrics are computed. All this information is saved in the database, in order to be able to evaluate different configuration and version of the application and provide a quantitative rational for the version deployed in production.

Each benchmark run will be saved in the table *benchmarks_run* with the following schema:

- `runId UUID PRIMARY KEY,`
- `startdate DATETIME DEFAULT CURRENT_TIMESTAMP,`
- `enddate DATETIME DEFAULT CURRENT_TIMESTAMP,`
- `hyperparameters: JSONB`

Each individual test, which is the equivalent of submitting one test pair to the application and retrieve the results set, is saved in the table named *benchmarks_run_test* with schema:

- `testid UUID PRIMARY,`
- `runid UUID REFERENCES benchmarks_run(runId),`
- `tpid UUID REFERENCES test_pairs(tpId),`
- `intention TEXT,`
- `keywords TEXT[],`
- `filters JSONB,`
- `rank INTEGER,`
- `runtime NUMERIC,`
- `extractiontime NUMERIC,`
- `buildingtime NUMERIC,`
- `querytime NUMERIC,`
- `sqlquery TEXT,`
- `resultsset JSONB,`
- `ranks INTEGER[],`
- `actualgroups TEXT[],`
- `kneepointresults JSONB`

The metrics calculated for each test pair test are stored in table *test_pair_metric_values* with schema:

- `id SERIAL PRIMARY KEY,`
- `runid UUID REFERENCES benchmarks_run(runId),`
- `testid UUID REFERENCES benchmarks_run(testId),`
- `metric TEXT,`

CC BY-SA

- `value NUMERIC`

Lastly the metrics calculated at the run level are stored in *run_metrics_values* with schema:

- `id SERIAL PRIMARY KEY,`
- `runId UUID REFERENCES benchmarks_run(runId),`
- `metric TEXT,`
- `value DECIMAL`

2.3 Test Pairs

In PaN-Finder, a test pair is the definition of a single test, which consists of the user prompt and the item or items that we are targeting in the results set returned by the system.
Originally, we planned to include in the PaN-Finder benchmark only *Single Positive* test pairs, which are defined as:

- Given a single user query, we expect that the results set contains the specified item.

Based on our initial interaction with the system, we assessed that the benchmark should be expanded to include also the following test pairs:

- *Single Negative*:
  Given a single user query, we expect that the results set does not contains the specified a item.
- *Multiple Positive*:
  Given a single user query, we expect that the results set contains the specified items, which can be two or more.

With further knowledge of the system behavior and additional considerations, we decided to stop the implementation of Single Negative and Multiple Positive tests pair, at least at this point in time. We opted to introduce new test pair types which requires a dedicated matching metric. The new test pair types are:

- Extended
- Comparative

We will described these test pair types in the related extended metrics section below.

2.4 Metrics

The metrics can be grouped in two overall categories: test pair metrics and run metrics. The test pair metrics are computed on the individual test pair, while the run metrics are computed on the whole run computing the average of the matching test pair metrics across all test pairs used in the run.
The test pair metrics, which are computed on the individual test pair, are depended on the type of test pair.

 CC BY-SA

Before diving in the definition of the PaN-Finder metrics, we first introduce the general metrics as they are found in the current literature. We will proceed to describe the custom version of the different metrics that have been adapted to our use case.

2.4.1 General Metrics

This section introduces different metrics that are generally applicable to information retrieval systems built with LLM. Some of the metrics listed below are commonly used in the industry and well defined in the current available literature. Some of them have been defined within the scope of this project, in order to better benchmark the project and its accuracy, and also to discover all the the subtle changes introduced by the tweaks applicable to the system.

2.4.1.1 Precision at K (Pc@K)

Precision at K measures how many items with the top K positions are relevant.
This metrics does not have rank awareness.
It's definition is:

$$Pc@K = \frac{N_{r@K}}{K}$$

where

- $N_{r@K}$ = Number of relevant items in the first K items of the results set
- K = number of best ranked items considered in the results set

Additional information can be found in the following resources:

- https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k
- https://keylabs.ai/blog/understanding-precision-at-k-p-k/
- https://krishnapullak.medium.com/understanding-precision-recall-and-f-score-at-k-in-recommender-systems-7146a0dce68e
- https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval)

2.4.1.2 Context Precision at K (CP@K)

Context Precision is a metric that evaluates whether all of the relevant items present in the results are ranked higher or not. Ideally all the relevant items must appear at the top ranks. This metrics has rank awareness, so the closer are the relevant items to the top, the higher is the score.
It's definition is:

$$CP@K = \frac{1}{N_{r@K}} \sum_{k=1}^{K} Pc@k * v_k$$

CC BY-SA

where:

- Pc@k = Precision @ k
- v_k = relevance indicator at rank k. It is 1 if item at rank k is relevant, 0 otherwise.
- $N_{r@k}$ = Number of relevant items in the top K items of the result set
- K = number of best ranked items considered in the results set,

Additional resources:

- https://docs.ragas.io/en/v0.1.21/concepts/metrics/context_precision.html
- https://docs.getdynamiq.ai/evaluations/metrics/predefined-metrics/context-precision
- https://www.confident-ai.com/docs/metrics/single-turn/contextual-precision-metric

2.4.1.3 Reciprocal rank at K (RR@K)

The reciprocal rank of a query response is the multiplicative inverse of the rank of the first correct answer.

$$RR@K = \frac{1}{r_1}$$

where

- r_1 = rank of the first relevant item.

Additional recources:

- https://en.wikipedia.org/wiki/Mean_reciprocal_rank

2.4.1.4 Presence at K (Ps@K)

This metric has been defined within the scope of the PaN-Finder project. The advantages of this metric are its easiness to compute and it is intuitive to understand.
This metric indicate if in the top K items there is at least one relevant item. It can be calculated using Precision @ K. This metric does not have rank awareness. Its definition is:

$$Ps@K = \begin{cases} 1 \, if \; Pc@K > 0 \\ 0 \, otherwise \end{cases}$$

2.4.1.5 Linear Zero Position at K (lzp@K)

This metrics average the weight across the number of relevant items in the top K items of the results set. IT has been defined within the PaN-Finder project. For each relevant item the metric assign a weight from 1 to 0 that linearly decrease with the rank. It zeros out at K+1. Relevant items that are at the end of the list are assigned a lower weight. This metric has rank awareness.

CC BY-SA

$$lzp@K = \frac{1}{N_{r@K}} \sum_{k=1}^{K} \left(1 - \frac{k-1}{K}\right) * v_k$$

where:

- $N_{r@K}$ = number of relevant items in the results set
- K = selected number of top items
- V_k = relevance indicator at rank k. It is 1 if item at rank k is relevant, 0 otherwise.

2.4.1.6 Exponential Zero Position at K (ezp@K)

This metrics average the weight across the number of relevant items in the top K items of the results set. As the previous metric, it has been defined within the PaN-Finder project. For each relevant item the metric assign a weight from 1 to 0 that exponentially decrease with the rank. It zeros out at K+1. This metric has rank awareness.

$$ezp@K = \frac{1}{N_{r@K}} \sum_{k=1}^{K} \left(\frac{e^{\frac{1}{S}(K-k)} - 1}{e^{\frac{K}{S}} - 1}\right) * v_k$$

where:

- $N_{r@K}$ = number of relevant items in the results set
- K = number of selected top items in the results set
- S = Curvature Scaling Constant with $S \in \mathbb{Z}_+^*$. Suggested value 1/3.
- V_k = relevance indicator at rank k. It is 1 if item at rank k is relevant, 0 otherwise.

2.4.1.7 Cumulative Gain at K (CG@K)

The cumulative gain is the sum of relevance weight of all relevant items among the top K results in the results set. The relevance weight, relevance for short, is a subjective number assigned indicating how relevant the item is. Usually it is assigned by users or empirically from metrics measured.
This metric is not rank aware.
The definition is:

$$CG@K = \sum_{k=1}^{K} R_k$$

where:

- R_k = relevance of item at position k

More information can be found in the following resources:

- https://en.wikipedia.org/wiki/Discounted_cumulative_gain

CC BY-SA

- https://www.evidentlyai.com/ranking-metrics/ndcg-metric

2.4.1.8 Discounted Cumulative Gain at K (DCG@K)

This metric introduce rank awareness to DCG@K by scaling the relevance score by a factor which is a logarithmic function of the position. We defined this metric as:

$$DCG@K = \sum_{k=1}^{K} \frac{R_k}{\log_2(k+1)}$$

where:

- R_k = relevance of item at position k

More information can be found in the following resources:

- https://en.wikipedia.org/wiki/Discounted_cumulative_gain
- https://www.evidentlyai.com/ranking-metrics/ndcg-metric

2.4.1.9 normalized Discounted Cumulative Gain at K (nDCG@K)

This metric normalizes DCG versus a DCG calculated on an ideal results set where the the items are in the correct rank. We defined this metric as:

$$nDCG@K = \frac{DCG@K}{eDCG@K}$$

where:

- R_k = relevance of item at position k

More information can be found in the following resources:

- https://en.wikipedia.org/wiki/Discounted_cumulative_gain
- https://www.evidentlyai.com/ranking-metrics/ndcg-metric

2.4.2 Metrics with 1 relevant item

During the initial testing of the PaN-Finder, we realized that the fastest way to start testing and the easiest way to create such tests was to check for the presence of only one selected item in the results set given a specific user prompt. We refer to such tests as Simple Positive test and the relative test pair a simple presence test pair. Such tests still consists the bulk of the tests conducted during our benchmarks.
Given that in Simple Presence tests we are looking to a single relevant item, we can simplify the metrics presented in the previous section and refactor their definitions as follow.

CC BY-SA

2.4.2.1 Precision at 1K (Pc@1K)

Precision at 1K, is derived from Pc@K substituting $N_{r@K}$ with 1 if the target item is present in the top K results, 0 otherwise.
The definition for Precision at 1K is:

$$Pc@1K = \begin{cases} \frac{1}{K} \, if \; 0 < r \leqslant K \leqslant N_{rs} \\ 0 \, otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N_{rs} = number of elements in the results set
- K = number of top results considered in the results set

With the current definition, this metric becomes irrelevant for our use case. It is equivalent to Presence@1K, which is easier to compute. We are using only Presence@1k.

2.4.2.2 Context Precision at 1K (CP@1K)

Context Precision at 1k is defined as:

$$CP@1K = \begin{cases} \frac{1}{r} \, if \; 0 < r \leq K \leq N_{rs} \\ 0 \, otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N_{rs} = Number of items in the results set
- K = number of top items considered in the results set,

2.4.2.3 Presence at 1K (Ps@1K)

Given that we have only one relevant result, the Ps@1K definition becomes:

$$Ps@1K = \begin{cases} 1 \, if \; 0 < r \leq K \leq N_{rs} \\ 0 \, otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N_{rs} = number of items in the results set
- K = number of top items considered in the results set,

CC BY-SA

2.4.2.4 Linear Zero Position at 1K, lzp@1K

This metrics assign a value from 1 to 0 that linearly decrease with the rank. It zeros out at K+1. This metric has rank awareness similar to CP@1K. Reworking the definition of lzp@K including the condition of 1 relevant item, its definition becomes:

$$lzp@1K = \begin{cases} 1 - \frac{r-1}{K} \; if \; 0 < r \leq K \leq N_{rs} \\ 0 \; otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N_{rs} = number of items in the results set
- K = selected number of top items

2.4.2.5 Exponential Zero Position at 1K (ezp@1K)

This metrics assign a value from 1 to 0 that exponentially decrease with the rank. It zeros out at K+1. This metric has rank awareness similar to CP@1K and its definition can be derived from ezp@K and is:

$$ezp@1N = \begin{cases} \frac{e^{\frac{1}{S}(K-r)} - 1}{e^{\frac{K}{S}} - 1} \; if \; 0 < r \leq K \leq N_{rs} \\ 0 \; otherwise \end{cases}$$

where

- r = rank of the desired document in the results set
- N_{rs} = number of items in the results set
- K = number of selected top items in the results set
- S = Curvature Scaling Constant with $S \in \mathbb{Z}_+^*$. Suggested value 1/3.

2.4.2.6 Discounted Cumulative Gain at 1K (DCG@1K)

With a single relevant item, discounted cumulative gain can be defined as:

CC BY-SA

$$DCG@1K = \begin{cases} \frac{R_r}{\log_2(r+1)} \; if \; 0 < r \leq K \leq N_{rs} \\ 0 \; otherwise \end{cases}$$

where:

- R_k = relevance of item at position k
- r = rank of the relevant item in the results set
- N_{rs} = Number of items in the results set
- K = number of top items considered in the results set,

2.4.2.7 normalized Discounted Cumulative Gain at 1K (nDCG@1K)

With a single relevant item, given the definition of DCG@1k and that the ideal position of the relevant item is R_e, the normalized Discounted Cumulative Gain can be defined as:

$$nDCG@1K = \frac{DCG@1K}{eDCG@1k} = \begin{cases} \frac{\frac{R_r}{\log_2(r+1)}}{\frac{R_{re}}{\log_2(r_e+1)}} \; if \; 0 < r, r_e \leq K \leq N_{rs} \\ 0 \; otherwise \end{cases}$$

where:

- R_k = relevance of item at position k
- R_{ke} = relevance of item at the expected position k_e
- r = rank of the relevant item in the results set
- r_e = expected rank of the relevant item in the results set
- N_{rs} = Number of items in the results set
- K = number of top items considered in the results set,

2.4.3 Single Positive Test pair metrics

As explained at the beginning of the section, the easiest tests is the simple positive test which is linked to single positive test pairs. Such test pairs are defined as a tuple of the relevant item expected in the results set, matched with a specific user prompt. This piece of information leads us to further simplify our single positive metrics by introducing the condition that the only relevant value of K in our use case is when the whole results set is considered. That translates by imposing K equal to the number of items in the results set N_{rs}, or simply N.
In the following section, we will introduce the metrics computed on each single positive test pairs. Some metrics previously presented have not been included as their contribution is not relevant

 BY-SA

anymore.

2.4.3.1 Context Precision at 1N (CP@1N)

This is context precision at the arity of the results set with at most 1 relevant item.
This metrics is also equivalent to the Reciprocal Rank @ N, RR@N, with 1 relevant item.
The simplified definition is:

$$CP@1N = \begin{cases} \frac{1}{r} \, if \; 0 < r \leq N \\ 0 \, otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N = Number of items in the results set

2.4.3.2 Presence at 1N (Ps@1N)

This metric indicate if the relevant item is present in the results set.

$$Ps@1N = \begin{cases} 1 \, if \; 0 < r \leq N \\ 0 \, otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N = Number of items in the results set

2.4.3.3 Linear Zero Position at 1N (lzp@1N)

Linear Zero Position is computed for 1 item across the whole results set with the assumption that the best position for item is the highest, or position 0. The further away from the highest rank, the lower is the relevancy, which follow a linear equation.

$$lzp@1N = \begin{cases} 1 - \frac{r-1}{N} \, if \; 0 < r \leq N \\ 0 \, otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N = number of items in the results set

CC BY-SA

2.4.3.4 Exponential Zero Position at 1N (ezp@1N)

Exponential Zero Position is computed for 1 item across the whole results set with the assumption that the best position for item is the highest, or position 0. The further away from the highest rank, the lower is the relevancy, which follow a negative exponential equation.

$$ezp@1N = \begin{cases} \dfrac{e^{\frac{1}{S}(N-r)} - 1}{e^{\frac{N}{S}} - 1} & if\ 0 < r \leq N \\ 0 & otherwise \end{cases}$$

where

- r = rank of the desired item in the results set
- N = number of items in the results set
- S = Curvature Scaling Constant with $S \in \mathbb{Z}_+^*$. Suggested value 1/3.

2.4.3.5 Linear Expected Position at 1N (lep@1N)

This metrics assigns a higher linear value between 0 and 1 if the relevant item is as closer to the expected position in the list.
At the moment, although included in the benchmarks, this metric is not relevant as we do not have a definitive knowledge of which is the expected position of the relevant item in the results set. We are hoping to be able to develop such knowledge, increase the meaningfulness of this metric and include it in our evaluation of the system as our knowledge increase.
The formal definition of the metric is:

$$lep@1N = \begin{cases} \dfrac{1}{N} |r_e - r| & if\ 0 < (r_e, r) \leq N \\ 0 & otherwise \end{cases}$$

where

- r = rank of the desired item in the results set
- N = number of items returned in the results set
- r_e = expected rank provided by domain knowledge experts or previous benchmarks

2.4.3.6 Exponential Expected Position at 1N (eep@1N)

This metrics assigns a higher exponential value between 0 and 1 if the relevant item is closer to the expected position in the list.
The same consideration highlighted for lep@1N are applicable for this metrics.

CC BY-SA

$$eep@1N = \begin{cases} \dfrac{e^{\frac{1}{S}|r_e - r|} - 1}{e^{\frac{N}{S}} - 1} & if\ 0 < r \leq N \\ 0 & otherwise \end{cases}$$

where

- r = rank of the desired item in the results set
- N = number of items returned in the results set
- r_e = expected rank provided by domain knowledge experts or previous benchmarks
- S = Curvature Scaling Constant with $S \in \mathbb{Z}_+^*$. Suggested value 1/3.

2.4.3.7 zero normalized Discounted Cumulative Gain at 1N (zNDCG@1N)

This is a ranking quality metric derived from the normalized Discounted Cumulative Gain. It compares the test pairs ranking to the ideal results set where the relevant item is at the top of the list.

$$znDCG@1N = \begin{cases} \dfrac{1}{\log_2(r+1)} & if\ 0 < r \leq N \\ 0 & otherwise \end{cases}$$

where

- r = rank of the desired item in the results set
- N = number of items returned in the results set

2.4.3.8 expected normalized Discounted Comulative Gain at 1N (eNDCG@1N)

As the previous metric, this one is a ranking quality metric derived from the normalized Discounted Cumulative Gain. It compares the test pairs ranking to the ideal results set where the relevant item is at the expected position in the list. As we explained for the other metrics where we have an expected position, this metric is not relevant at the moment because we have not yet develop the correct knowledge and methodology to define the expected position. The hope is that as we become more familiar with the system and more benchmark data is available, we can assign an expected position to the selected items.

$$enDCG@1N = \begin{cases} \dfrac{\log_2(r_e+1)}{\log_2(r+1)} & if\ 0 < r \leq N \\ 0 & otherwise \end{cases}$$

where

CC BY-SA

- r = rank of the desired item in the results set
- N = number of items returned in the results set
- r_e = expected rank provided by domain knowledge experts or previous benchmarks

2.4.4 Single Negative Test Pairs Metrics

During the developing of the PaN-Finder application and benchmarks, we were planning to include single negative tests and test pairs. The single negative test pairs are tuple with a user query and the reference to an item that should not be in the results set. After much consideration, we decided not to continue using such metric and include them directly in the extended metric, introduced below.
For the sake of documenting the process, we report the original idea behind the metric linked to the single negative test pair.
The only metric that has relevance in this case is the inverse of the Presence @ K, which we called Absence @ K, simplified with 1 relevant item and n items in the results set.

2.4.4.1 Absence at 1N (Ab@1N)

This metric indicates if the relevant item is not present in the results set. The definition is:

$$Ps@1N = \begin{cases} 0 \, if \; 0 < r \leq N \\ 1 \, otherwise \end{cases}$$

where

- r = rank of the relevant item in the results set
- N = Number of items in the results set

2.4.5 Extended Test Pairs Metrics

After lunching the beta version of the PaN-Finder and further developing the overall system with the introduction of the knee-point algorithm (please see deliverable 2.b), we performed a system review and new round of manual testing. The conclusion reached was that basic metrics and basic test pairs were still useful but not enough to cover some use cases that we found based on new user queries that we collected. We decided to introduce additional extended metrics to be able to test this new cases.

The first type of new test pairs include a user query, the item considered and the group it expected to belong to: relevant, suggested and non relevant. The metric measure if the results set of the system lists the selected item in the correct group or not. Actual group vs expected group decide the metric value, such value is defined in the hyper parameters of the benchmark run.

CC BY-SA

An additional case discovered is when the user provides information about a query and two or more documents that should be presented in a specific order relative to each other. A new test pair type and relative metric has been introduced to measure the performances and accuracy of the system.
Each new extended metric can be computed only if the extended test pair of the correct type is provided. In the following paragraph, we will introduced the new extended test pair together with the relative metric definition.

2.4.5.1 Extended Presence at 1N (EP@1N)

This metric indicate if the relevant item is in the correct group indicated in the test pair. The metric provides a score based on the weight specified in the hyper parameters of the benchmark run.
The formal metric is defined as:

$$EP@1N = w_{Ga,Gb}$$

where:

- $w_{Ga,Gb}$ is the defined scored for the selected item when belonging to actual group Ga while expected to belong to expect group Gb
- Ga is the actual group that the select item belongs to in the results set
- Ge is the expected group that is indicated in the extended presence test pair

This metric requires the definition of the weights matrix which assign the scores for each case, for example the score if the item was supposed to be in the HR group but was listed in the LR one or was no present at all. The three groups are: HL (High Relevance or Relevant), LR (Low Relevance or Suggested) and NP (Non Present or the item is not in the results set)
One example of EP score matrix is the following:

	Actual Group *Ga*			
Expected Group *Ge*		HR	LR	NP
	HR	1	0.25	-1
	LR	0.25	1	-1
	NP	-1	-1	1

CC BY-SA

This example penalize more a HR item wrongly assigned to the NP group than wrongly assigned to the LR group. To summarize, a switch between group HR and LR is less important than between groups HR or LR and NP.
The EP weights matrix is part of the parameter space defined in the benchmark hyper parameter files.
The test pair associated with this metric is defined as following:

$$TP_{EP} = (t, q, doi, g_e)$$

where:

- t is the type of the test pair, in this case EP
- q is the query submitted to the system
- doi is the document that we are targeting
- g_e is the expected group this document is expected to belong to, which can have one of the following values:
  1. HR = High Relevance. In the FE, it is shown as a relevant document
  2. LR = Low Relevance. In the FE, it is shown as a suggested document
  3. NP = Not Present. The document should not in the result set

2.4.5.2 Comparative Relative Presence @ N (CRP@N)

This metric provide a score which indicate if the listed items are in the correct relative position compared to each other. The value is 1 if all the items listed are in the correct order, 0 if none of them are in the correct order.
The metric is defined as:

$$CRP@N = \frac{1}{(I-1)} \sum_{i=1}^{I-1} \begin{cases} 1 \, if \; 0 < r_{di} < r_{d(i+1)} \leq N \\ 0 \, otherwise \end{cases}$$

where:

- I is the number of items listed in the test pair
- r_{di} is the rank of item listed in position i in the test pair
- $r_{d(i+1)}$ is the rank of item listed in position i+1 in the test pair
- N is the number of items in the results set

The test pair associated with this metric is defined as following:

CC BY-SA

$$TP_{CP}=(t, q, doi_1, doi_2, \ldots, doi_I)$$

where

- t is the type of the test pair, in this case CP
- q is the query submitted to the system
- $doi_1, doi_2, \ldots, doi_I$ is the list of items that should be found in the results set in the exact relative position
- I is the number of selected items listed in the test pair

2.4.6 Time metrics

For each test pairs, independently of the type, we collect time metrics, which are saved directly in the test pair test record. They are:

- *Total Execution Time*, TET
  This is the total time that the test took from prompt submission to returning the results set.

- *Prompt Extraction Time*, PET
  This is the time elapsed to extract the main components from the user query. The components are: the intention, the kewords and the filters.

- *Query Building Time*, QBT
  This is the time elapsed to build the full hybrid query.

- *Sql Query Time*, SQT
  This is the time elapsed to execute the query on the database and return the results set

2.4.7 Run metrics

For each run, the mean value of all the metrics are compute.
The metrics for the run are defined as follow:

$$\overline{M}(H)=\overline{M}(T, N)=\frac{1}{card(T)}\sum_{t\in T} M@N(t)$$

where:

- M is the metric considered
- H are the hyper-parameters of the run
- T is the set of test pairs considered

CC BY-SA

- N is the number of items returned in the results set

The following table lists all the metrics that should be computed for each run, after the individual test pair metrics are computed.
Given the hyper parameters, the run's metrics will be named according the following naming convention:

*t*M*mm*

where
- *t* is the test pair sets used selected to compute the run metric. At the time of this writing, the possible options are:
  - c for comprehensive, which includes all the test pairs,
  - s for synthetic, which includes only the synthetic pairs AI generated, and
  - e which contains all the test pairs provided by real users and expert domains.
- *mm* is the selected test pair metric, which can be any of the following: CP, Ps, a so forth

The extended name of the run metric should be:

*t*M*mm@n(h)*

where:
1. *n* is the number of items returned (or arity) of the results set
2. *h* is the rest of the hyper parameters that are not used directly in the calculation of the metric but are important in the computation of the score.

The values of n and h are explicitly defined in the hyper parameter of the specific run, so we simplify the naming and use the previous version.

Table of all the run metrics provided

Test Pairs Set				Name	Comprehensive	Synthetic	Expert
Test Pairs Metric name				Symbol	T_C	T_S	T_E
Full	*Short*	*Selected*					
Context Precision	CP	y		MCP	cMCP	sMCP	eMCP
Presence	Ps	y		MPs	cMPs	sMPs	eMPs
Linear zero position	lzp	n		Mlzp	cMlzp	sMlzp	eMlzp
exponential zero position	ezp	n		Mezp	cMezp	sMezp	eMezp

CC BY-SA

Linear expected position	lep	n		Mlep	cMlep	sMlep	eMlep
Exponential expected position	eep	n		Meep	cMeep	sMeep	eMeep
zero normalized Discounted Comulative Gain	znDCG	n		MznDCG	cMznDCG	sMznDCG	eMznDCG
expected normalized Discounted Comulative Gain	enDCG	n		MenDCG	cMenDCG	sMenDCG	eMenDCG
Extended Presence	EP	y*		MEP	cMEP	sMEP	eMEP
Comparative Presence	CP	y*		MCP	cMCP	sMCP	eMCP

At the moment, we can not draw any conclusion if all the metrics mentioned above are valuable and informative to estimate the accuracy of system. Our approach has been to compute all of them until we can make an informed decision about which are the most informative metrics.

In addition to the run metrics mentioned in the table above, we will also compute the mean execution time for each run, which is dependent from the hyper parameters set for the run itself and the hardware where the benchmark has been run on. Following is the list of run time metrics:

1. Mean Total Execution Time, MTET@K
2. Mean Prompt Execution Tim, MPET@K
3. Mean Sql Execution Time, MSET@K

CC BY-SA

3 Resources

1. https://keylabs.ai/blog/understanding-precision-at-k-p-k/
2. https://krishnapullak.medium.com/understanding-precision-recall-and-f-score-at-k-in-recommender-systems-7146a0dce68e
3. https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval)
4. https://medium.com/@med.el.harchaoui/rag-evaluation-metrics-explained-a-complete-guide-dbd7a3b571a8
5. https://medium.com/@plthiyagu/comprehensive-evaluation-metrics-for-retrieval-augmented-generation-rag-a846ec355c86
6. https://www.confident-ai.com/blog/rag-evaluation-metrics-answer-relevancy-faithfulness-and-more
7. https://www.confident-ai.com/blog/the-ultimate-llm-evaluation-playbook
8. https://www.confident-ai.com/blog/llm-evaluation-metrics-everything-you-need-for-llm-evaluation
9. https://www.evidentlyai.com/ranking-metrics/precision-recall-at-k
10. https://en.wikipedia.org/wiki/Discounted_cumulative_gain
11. https://www.evidentlyai.com/ranking-metrics/ndcg-metric
12. https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/
13. https://oecd.ai/en/catalogue/metrics/context-precision-1
14. https://en.wikipedia.org/wiki/Mean_reciprocal_rank
15. https://docs.ragas.io/en/v0.1.21/concepts/metrics/context_precision.html
16. https://docs.getdynamiq.ai/evaluations/metrics/predefined-metrics/context-precision
17. https://www.confident-ai.com/docs/metrics/single-turn/contextual-precision-metric

CC BY-SA

4 Funding

The PaN-Finder (Photon and Neutron federated knowledge finder) project is founded by the OSCARS project.
The authors acknowledge the OSCARS project, which has received funding from the European Commission's Horizon Europe Research and Innovation programme under grant agreement No. 101129751

5 Licensing

This document is release under ***CC BY-SA 4.0*** license.
More information about the license is available at https://creativecommons.org/licenses/by-sa/4.0